

School of Electrical and Computer Sciences
Indian Institute of Technology, Bhubaneswar

LIPSI Processor

Author: Yashmith Gali

Roll Number: 24EC01019

Supervised by: Dr. Srinivas Boppu

Affiliation: IIT Bhubaneswar

Branch: ECE

Date: 16th May 2026

Project Link: <https://github.com/GYN26K/Lipsi-8bit-processor>

Table of Contents

1.	Introduction	3
2.	The Design	3
3.	The Datapath	4
4.	The Instruction Set	5
5.	Working of Instructions (FSM State Analysis)	7
6.	Verilog Implementation	11
7.	Fibonacci Program	13
8.	Possible Improvements	15
9.	Difficulties Faced	15
10.	Conclusion	16
11.	References	16

1. Introduction

This report presents the design and Verilog implementation of the Lipsi processor, a lightweight 8-bit accumulator-based microcontroller optimised for FPGA deployment. Originally proposed by Martin Schoeberl in the paper "Lipsi: Probably the Smallest Processor in the World", Lipsi targets embedded peripheral control tasks and finite-state-machine logic inside a larger system-on-chip, where minimal resource consumption is paramount.

Lipsi follows the von Neumann architecture: both the program code and the data it processes reside in a single 512-byte block RAM, partitioned at the midpoint. This unified memory design eliminates the need for separate instruction and data buses, keeping the hardware footprint extremely small and making Lipsi well-suited for FPGA block-RAM primitives.

Lipsi is a multi-cycle CPU. Every instruction is executed over multiple clock cycles — most requiring approximately two to three cycles. Each cycle performs one of the classical pipeline stages: instruction fetch, decode and operand read, or execute and write-back. Reusing the same functional units (ALU, memory, accumulator) across these stages avoids duplicating hardware and further reduces area.

This implementation extends the baseline architecture described in the original paper with an integrated four-digit seven-segment display driver and a Fibonacci number computation program, allowing the processor result to be observed in real time on an FPGA board.

2. The Design

Lipsi is an 8-bit processor designed around an accumulator-based execution model optimised specifically for FPGA block RAMs. The central design constraint is that a single block RAM must serve both instruction storage and data storage simultaneously, eliminating any need for dedicated register files implemented in flip-flops or distributed RAM.

The on-chip memory consists of 512 bytes (4096 bits). It is divided equally into two 256-byte regions, selected by the most-significant bit of the 9-bit memory address: logic 0 selects the data region, and logic 1 selects the program region. Within the data region, the lowest 16 bytes act as a general-purpose register file, addressable directly by the 4-bit register field in each instruction. The remaining 240 data bytes are available for general-purpose storage via indirect addressing.

Because the same block RAM is time-shared between instruction fetch and data read/write, the architecture is inherently sequential and non-pipelined. This is a deliberate design trade-off: pipelining would require either a second memory port or additional buffering, both of which increase resource usage. The sequential model keeps the control logic simple and the hardware small enough to fit multiple Lipsi instances on a single mid-range FPGA.

3. The Datapath

The Lipsi datapath consists of five principal components: the program counter (PC), the unified memory block, the arithmetic logic unit (ALU), the accumulator register (A), and a network of multiplexers and an adder that route data between these components. The control unit, implemented as an external finite state machine, generates the select and enable signals that govern the datapath but is not itself part of the datapath.

3.1 Program Counter

The PC is an 8-bit clocked register. On each rising clock edge it loads the value presented at its `pc_in` port when `pc_en` is asserted; otherwise it holds its current value. On reset it is forced to 0x00. In normal sequential

execution the `pc_in` value comes from the `add_1` incrementer ($PC + 1$). For branch instructions it is overridden with the branch target supplied from the read-data bus or from the accumulator, selected by the 4-to-1 PC multiplexer.

3.2 Accumulator Register

The accumulator is the sole explicit programmer-visible register. It stores the result of every ALU operation and serves as both source and destination for arithmetic, logical, shift, load, and store instructions. Its 8-bit clocked register is enabled by the `acc_en` control signal, which is asserted only in the states that commit a result (S1, S5, S8, and the shift path through S0).

3.3 ALU

The ALU is a purely combinational 8-bit unit. It receives the current accumulator value and the data read from memory (`rd_data`), selects the appropriate operation based on the 4-bit `alu_ctrl` signal, and drives the `acc_in` bus. Because the control unit latches the `alu_ctrl` value one cycle early (`alu_ctrl_store`), the ALU output is stable when the accumulator samples it on the following rising edge.

3.4 Memory

The 512-byte synchronous RAM is shared between program and data. The fetch signal forms the MSB of the 9-bit read address, routing accesses to the instruction region when asserted and to the data region when deasserted. The `small_addr` signal further constrains the lower address bits to [3:0] when accessing the 16-byte register file, simplifying the decode logic for register-mode instructions. Write-enable (`wr_en`) is derived from the `wr_addr` control output of the FSM, gated appropriately by the current state.

3.5 Multiplexers

Two 2-to-1 multiplexers select the write address and write data respectively. A 4-to-1 multiplexer selects the PC input from four sources: $PC+1$ (sequential), accumulator (branch and link return), `rd_data` (branch target), or zero (reset path). The `mux_pc_rd[1:0]` control field determines which source is active each cycle.

4. The Instruction Set

The Lipsi instruction set contains 12 instructions encoded in one or two bytes. Single-byte instructions carry the register number or shift sub-operation in the lower 4 bits. Two-byte instructions (ALU immediate, branch variants, and I/O) fetch a second byte from the instruction memory in the next cycle to supply the immediate constant or branch target address.

In the table below, *A* denotes the accumulator; *f* is an ALU function selector; *m*[] refers to a memory location; *r* is a 4-bit register number (0–15); *n* is an 8-bit immediate constant; *a* is an 8-bit branch target address; and *IO* denotes an input/output peripheral.

Table 1: Lipsi Instruction Set

Encoding	Instruction	Meaning	Operation
0fff rrrr	f rx	ALU register	$A = A \text{ f } m[r]$
1000 rrrr	st rx	store A into register	$m[r] = A$
1001 rrrr	brl rx	branch and link	$m[r] = PC, PC = A$
1010 rrrr	ldind (rx)	load indirect	$A = m[m[r]]$

Encoding	Instruction	Meaning	Operation
1011 rrrr	stind (rx)	store indirect	$m[m[r]] = A$
1100 -fff nnnn nnnn	fi n	ALU immediate	$A = A \text{ f } n$
1101 --00 aaaa aaaa	br	branch	$PC = a$
1101 --10 aaaa aaaa	brz	branch if A is zero	$PC = a \text{ if } A == 0$
1101 --11 aaaa aaaa	brnz	branch if A is not zero	$PC = a \text{ if } A != 0$
1110 --ff	sh	ALU shift	$A = \text{shift}(A)$
1111 aaaa	io	input and output	$IO = A, A = IO$
1111 1111	exit	exit for the tester	$PC = PC \text{ (halt)}$

Most instructions complete in two clock cycles (S0 decode + one execute state). The load-indirect and store-indirect instructions require three cycles because a second memory access is needed to resolve the pointer. Branch instructions also take two cycles: in S0 the branch condition is evaluated and the next byte is pre-fetched as the target address; in S9 (branch taken) or S10 (branch not taken) the PC is updated accordingly.

Table 2: ALU Operations

ALU Control	Name	Operation	Description
4'b1000	ADD	$A = A + rd_data$	Addition
4'b1001	SUB	$A = A - rd_data$	Subtraction
4'b1010	ADC	$A = A + rd_data + c$	Add with Carry
4'b1011	SBB	$A = A - rd_data - c$	Subtract with Borrow
4'b1100	AND	$A = A \& rd_data$	Bitwise AND
4'b1101	OR	$A = A rd_data$	Bitwise OR
4'b1110	XOR	$A = A \wedge rd_data$	Bitwise XOR
4'b1111	LOAD	$A = rd_data$	Load from memory
4'b0100	LSL	$A = A \ll 1$	Logical Shift Left
4'b0101	LSR	$A = A \gg 1$	Logical Shift Right
4'b0110	ASL	$A = A \lll 1$	Arithmetic Shift Left
4'b0111	ASR	$A = A \ggg 1$	Arithmetic Shift Right

5. Working of Instructions — FSM State Analysis

The control unit is implemented as a Moore finite state machine with 12 states (S0–S11). In a Moore machine the outputs depend only on the current state, not on the inputs. This guarantees glitch-free control signals and simplifies timing analysis. The next-state logic is purely combinational; the state register updates on every rising clock edge.

The FSM uses three always blocks: one clocked block advances the state register, one combinational block computes next_state from the current state and the instruction/accumulator inputs, and one combinational block drives all output control signals from the current state alone.

5.1 State Table

Table 3: FSM State Description and Control Signals

State	Role	Main Action	pc_en	acc_en	fetch	mux_pc_rd
S0	Instruction Decode	Decodes opcode, selects ALU function, determines next state	0	0	0	varies
S1	ALU Register Execute	Writes ALU result to accumulator, increments PC, fetches next instr	1	1	1	00
S2	Store Complete	Stores accumulator value into register, increments PC	1	0	1	00
S3	Branch and Link	Saves return address to register, updates PC from accumulator	1	0	1	01
S4	Indirect Load Phase 1	Reads pointer/effective address from register area	0	0	0	01
S5	Indirect Load Phase 2	Loads final operand from effective address into accumulator	1	1	1	00
S6	Indirect Store Phase 1	Reads pointer/effective address from register area	0	0	0	—
S7	Indirect Store Phase 2	Stores accumulator to memory at indirect address, increments PC	1	0	1	00
S8	Immediate ALU Execute	Applies ALU op with immediate constant, writes result, increments PC	1	1	1	00
S9	Branch Taken	Loads branch target into program counter	1	0	1	10
S10	Branch Not Taken	Advances PC normally, continues sequential execution	1	0	1	00
S11	Halt	Processor locks permanently until reset	0	0	1	11

5.2 State-by-State Description

S0 — Instruction Decode

S0 is the decode hub of the FSM. The instruction byte held in `rd_data` is examined every cycle the processor spends here. If `instruction[7]` is 0, the upper nibble encodes the ALU function and the lower nibble identifies the source register, so the FSM proceeds to S1. Otherwise, `instruction[7:4]` selects the instruction group and drives the transition to the appropriate execute state. In S0, `pc_en` and `acc_en` are deasserted so neither register is disturbed during decoding.

For the shift instruction group (4'b1110) the entire operation completes within S0 itself: `acc_en` and `pc_en` are both asserted in the same cycle, allowing a one-cycle shift that returns immediately to S0. For the halt group (4'b1111) the FSM unconditionally enters S11.

S1 — ALU Register Execute

S1 finalises a register ALU operation. The ALU result (captured in `alu_ctrl_store` from the previous cycle) is written into the accumulator (`acc_en = 1`). Simultaneously, `pc_en` and `fetch` are asserted so the PC advances to PC+1 and the next instruction is fetched from the instruction region. The FSM returns to S0 in the following cycle.

S2 — Store Complete

S2 commits a store instruction. The accumulator value is written to the selected register address in the data memory. The accumulator is not modified (`acc_en = 0`). PC increments and the next instruction is fetched before returning to S0.

S3 — Branch and Link

S3 handles the branch-and-link instruction. The current PC (return address) is written into the register specified by the instruction (`wr_data = 1` selects `pc_out` through the write-data mux). `mux_pc_rd = 2'b01` routes the accumulator value into the PC, effecting the indirect jump. The processor then returns to S0 at the new address.

S4 and S5 — Indirect Load

Indirect load requires two extra cycles beyond the decode. In S4 the effective address is read from the register file (`small_addr = 0` allows a full-range address lookup). In S5 the data at that effective address is loaded into the accumulator (`acc_en = 1`) and the PC is advanced, completing the instruction.

S6 and S7 — Indirect Store

Indirect store mirrors the load sequence. In S6 the pointer is read from the register file (`small_addr = 0`, `wr_addr = 1` so the read address is the register's value). In S7 the accumulator is written to the resolved target address and the PC increments.

S8 — Immediate ALU Execute

S8 executes an ALU immediate instruction. The immediate operand was pre-fetched from the second instruction byte in S0 (`fetch = 1`, `pc_en = 1` advanced the PC past the opcode byte). In S8 the ALU result is committed to the accumulator and the PC moves to the instruction following the immediate byte.

S9 — Branch Taken

S9 is entered when a branch condition is met (unconditional branch, BRZ with `A==0`, or BRNZ with `A!=0`). `mux_pc_rd = 2'b10` selects `rd_data` (the pre-fetched branch target address) as the new PC value, redirecting program flow.

S10 — Branch Not Taken

S10 is entered when the branch condition fails. `mux_pc_rd = 2'b00` selects PC+1, so sequential execution continues from the instruction immediately after the two-byte branch instruction.

S11 — Halt

S11 is a terminal absorbing state. Once entered (via the exit instruction 0xFF), the FSM never leaves S11 until an external reset is applied. `pc_en` and `acc_en` remain deasserted, freezing the processor state.

5.3 Instruction-Level Execution Flow

The table below summarises the state sequence for each instruction type:

Instruction	States Used	Total Cycles (incl. fetch in S0)
ALU Register (0fff rrrr)	S0 → S1 → S0	2
Store (1000 rrrr)	S0 → S2 → S0	2
Branch and Link (1001)	S0 → S3 → S0	2
Load Indirect (1010)	S0 → S4 → S5 → S0	3
Store Indirect (1011)	S0 → S6 → S7 → S0	3
ALU Immediate (1100)	S0 → S8 → S0	2 (2nd byte fetched in S0)
Branch (1101 --00)	S0 → S9 → S0	2
BRZ — taken (1101 --10)	S0 → S9 → S0	2
BRZ — not taken	S0 → S10 → S0	2
BRNZ — taken (1101 --11)	S0 → S9 → S0	2
BRNZ — not taken	S0 → S10 → S0	2
ALU Shift (1110)	S0 (single-cycle)	1
Halt / Exit (1111 1111)	S0 → S11	∞ (until reset)

6. Verilog Implementation

The design is structured as a hierarchy of 14 Verilog modules, each responsible for a single well-defined function. This modular decomposition improves readability, simplifies simulation, and isolates faults during debugging.

Table 4: Verilog Module Summary

Module	File	Description
ctrl_unit	ctrl_unit.v	12-state Moore FSM — decodes instructions and drives all control signals
alu	alu.v	8-bit combinational ALU supporting 12 operations including shifts and carry
accumulator	accumulator.v	8-bit clocked accumulator register with synchronous reset
pc	pc.v	8-bit program counter with enable and synchronous reset
add_1	add_1.v	Combinational PC+1 incrementer
memory	memory.v	512-byte unified block RAM for both instructions and data
mux2x1	mux2x1.v	8-bit 2-to-1 multiplexer for write address and data selection
mux4x1	mux4x1.v	8-bit 4-to-1 multiplexer for PC input source selection
cpu	cpu.v	Top-level datapath instantiating all processor modules
clk_div	clk_div.v	Clock divider — scales 100 MHz down to 1 Hz for visible display
bin_to_bcd	bin_to_bcd.v	Binary to BCD converter (ones, tens, hundreds, thousands)
display_mux	display_mux.v	4-digit seven-segment display multiplexer with refresh counter
seg7_decoder	seg7_decoder.v	7-segment encoding for digits 0–9
main_module	main_module.v	Top-level FPGA wrapper connecting CPU, clock divider, and display

6.1 Control Unit (ctrl_unit.v)

The control unit is the most complex module. It contains three always blocks. The first, clocked, latches `alu_ctrl` into `alu_ctrl_store` one cycle ahead of the execute state so the ALU result is stable when the accumulator samples it. The second, also clocked, advances the state register. The third and fourth (both combinational) compute `next_state` and drive the output control signals.

Key design points extracted from the Verilog:

- S0 inspects `instruction[7]` first. A zero MSB immediately implies an ALU register instruction.
- Branch condition evaluation (BRZ / BRNZ) happens in S0 itself, using the current accumulator value A, so the correct S9/S10 next state is selected in one cycle.

- The shift instruction (1110) is the only instruction that sets both `pc_en` and `acc_en` inside `S0`, making it a true single-cycle operation.
- `S11` is self-looping: `next_state = S11` regardless of inputs, providing a clean halt.

6.2 ALU (`alu.v`)

The ALU is a 12-operation combinational unit. Arithmetic operations (ADD, SUB, ADC, SBB) propagate a carry/borrow bit via the internal register `c`. Logical operations (AND, OR, XOR) are bitwise. The LOAD operation (4'b1111) bypasses the accumulator and writes `rd_data` directly. Shift operations use Verilog's native shift operators: `<<` and `>>` for logical shifts, `<<<` and `>>>` for arithmetic (sign-extending) shifts.

```
// ALU core - selected excerpts
4'b1000: {c, acc_in} = acc_out + rd_data;    // ADD
4'b1001: {c, acc_in} = acc_out - rd_data;    // SUB
4'b1100: acc_in = acc_out & rd_data;         // AND
4'b0100: acc_in = acc_out << 1'b1;          // LSL
4'b0111: acc_in = acc_out >>> 1'b1;         // ASR
default: acc_in = acc_out;                  // hold
```

6.3 Memory (`memory.v`)

The synchronous 512-byte RAM initialises on reset with the Fibonacci program pre-loaded into both the data region (registers `r0`–`r4`) and the instruction region (addresses 256–271). On reset, `rd_data` is also preloaded with the first instruction byte (`mem[256] = 0x70`) so that `S0` can begin decoding immediately without a separate initialisation cycle.

During normal operation, the `fetch` bit forms the MSB of the 9-bit address. When `fetch` is high the RAM returns instruction bytes; when low and `small_addr` is high, only the lower 4 address bits are used, constraining accesses to the 16-byte register file. When `small_addr` is low, the full lower 8 bits are used, enabling indirect accesses anywhere in the data region.

6.4 Display Subsystem

Three modules implement the seven-segment display output visible on the FPGA board. The `clk_div` module divides the 100 MHz board clock down to 1 Hz using a 27-bit counter, producing a slow clock that drives the CPU at human-visible speed. The `bin_to_bcd` module converts the 8-bit accumulator result to four BCD digits using integer division and modulo. The `display_mux` module uses a 16-bit refresh counter to cycle through four digit positions at roughly 1.5 kHz, illuminating one digit at a time so that all four appear simultaneously to the human eye.

7. Fibonacci Program

The processor is pre-loaded with an assembly-level Fibonacci program that computes the N-th Fibonacci number. The value of N is supplied as a 4-bit input port to the memory module and stored in register r3 at reset time. The result accumulates in r2 and is loaded into the accumulator immediately before the exit instruction, so the seven-segment display shows the final Fibonacci number.

7.1 Register Allocation

Register	Address	Purpose	Init
r0	mem[0]	Previous Fibonacci number (F_{n-1})	0
r1	mem[1]	Current Fibonacci number (F_n)	1
r2	mem[2]	Temporary result (F_{n+1})	0
r3	mem[3]	Loop counter (counts down from N to 0)	N
r4	mem[4]	Constant 1 — used for counter decrement	1

7.2 Algorithm

The program implements the iterative Fibonacci recurrence:

```

F0 = 0;  F1 = 1;
repeat N times:
    temp = F0 + F1;
    F0   = F1;
    F1   = temp;
result in F1 (stored via r2 before exit)

```

7.3 Instruction-by-Instruction Trace

Table 5: Fibonacci Program Memory Map

Address	Hex Value	Description
mem[0]	0x00	r0 = 0 (previous Fibonacci number)
mem[1]	0x01	r1 = 1 (current Fibonacci number)
mem[2]	0x00	r2 = 0 (temporary result)
mem[3]	{ 4 'b0, N }	r3 = N (loop counter)
mem[4]	0x01	r4 = 1 (constant for decrement)
mem[256]	0x70	Load r0 into accumulator A
mem[257]	0x01	A = A + r1 (add previous + current)
mem[258]	0x82	Store A into r2 (save temp result)
mem[259]	0x71	Load r1 into A
mem[260]	0x80	Store A into r0 (r0 = old r1)
mem[261]	0x72	Load r2 into A
mem[262]	0x81	Store A into r1 (r1 = new Fibonacci)

Address	Hex Value	Description
mem[263]	0x73	Load loop counter r3 into A
mem[264]	0x14	A = A - r4 (decrement counter)
mem[265]	0x83	Store updated counter into r3
mem[266]	0xd2	Branch if A == 0 (loop exit)
mem[267]	0x0e	Branch target address (exit address)
mem[268]	0xd3	Branch if A != 0 (loop continue)
mem[269]	0x00	Branch target address (loop start = 256)
mem[270]	0x72	Load final Fibonacci value (r2) into A
mem[271]	0xff	Exit / halt

7.4 Loop Control

The loop counter in r3 is decremented by 1 (using register r4 which holds the constant 1) at each iteration. After the decrement the updated counter is tested: if the accumulator is zero (BRZ at address 266), the branch target 0x0e (decimal 14 — but the actual target resolves to address 270 after the offset) exits the loop; otherwise BRNZ at address 268 branches back to address 0x00 (256), restarting the loop body. After the loop, mem[270] loads r2 into the accumulator and mem[271] halts the processor.

8. Possible Improvements

The following enhancements would increase the performance, flexibility, and usability of the current implementation:

1. Cycle count reduction: Introducing dedicated per-instruction states rather than shared execute states would enable finer timing control and reduce the clock cycles per instruction for common operations.
2. Parallel memory access: Store-type instructions could exploit parallelism in memory read and write by overlapping data movement, reducing effective CPI for store-heavy workloads.
3. Efficient encoding: Subtraction could reuse the adder with a complemented operand rather than maintaining a separate subtract path, reducing logic-element count.
4. Custom instruction support: Application-specific instructions could be added to accelerate common computation patterns, particularly useful for peripheral control loops.
5. UART bootloader: A lightweight UART-based bootloader would allow machine code to be uploaded to the FPGA at runtime, removing the need to re-synthesise the design every time the program changes.
6. Pipelined fetch: A two-entry instruction buffer would overlap instruction fetch with execution, recovering one cycle per instruction without requiring a true pipeline.

9. Difficulties Faced

Designing a multi-cycle processor from scratch involved navigating several challenging aspects of digital design, particularly for someone early in the learning curve of processor architecture.

The most significant challenge was constructing the finite state machine correctly. Mapping each instruction to the right sequence of states, and ensuring that the control signals in every state matched the exact requirements of that instruction phase, required careful analysis of the datapath. Early versions of the FSM produced incorrect results because of subtle errors in the `wr_addr` and `wr_data` mux selections, which were difficult to trace without systematically stepping through each instruction in simulation.

Assigning the correct values to the seven control outputs (`mux_pc_rd`, `wr_addr`, `wr_data`, `pc_en`, `acc_en`, `fetch`, `small_addr`) for every state demanded a clear mental model of what the datapath must do at each clock edge. A single incorrect bit could cause the wrong memory location to be read, the PC to advance prematurely, or the accumulator to be overwritten with garbage.

The indirect addressing instructions (load indirect and store indirect) were particularly tricky because they require a three-cycle execution sequence and the intermediate address must survive correctly from S4/S6 into S5/S7. Getting the `small_addr` control correct to switch between register-file mode and full-range mode at the right cycle was error-prone.

Finally, integrating the seven-segment display subsystem with the slow-clocked CPU required care to avoid metastability between the fast display refresh clock and the slow CPU clock, solved by driving the display mux from the original fast clock while the CPU runs on the divided clock.

10. Conclusion

The design and Verilog implementation of the Lipsi 8-bit processor provided deep, practical insight into processor architecture, finite state machine based control, and hardware description language design methodology. Despite the minimalist constraints — a single 512-byte block RAM, one accumulator, and a 12-state FSM — the processor is capable of running non-trivial programs such as the Fibonacci computation demonstrated here.

The modular structure of the Verilog implementation, with separate modules for the ALU, accumulator, program counter, memory, control unit, and display subsystem, made it possible to isolate and debug each function independently before integrating the complete system. This discipline of modular design is one of the most transferable lessons from the project.

The difficulties encountered — particularly in FSM control signal assignment and indirect addressing — reinforced the importance of systematic simulation and careful datapath analysis before committing to synthesis. The experience has laid a strong foundation for tackling more advanced processor designs, including pipelined architectures, wider data paths, and RISC-style register files.

11. References

7. Martin Schoeberl, "Lipsi: Probably the Smallest Processor in the World", Architecture of Computing Systems (ARCS), Springer, 2018.
8. Original Lipsi Chisel reference implementation: <https://github.com/schoeberl/lipsi>
9. This project's Verilog implementation: <https://github.com/GYN26K/Lipsi-8bit-processor>
10. Verilog HDL Reference Manual, IEEE Std 1364-2005.
11. FPGA Block RAM Design Guidelines, Xilinx UG473.